

CodeSwarm

Solution Design Document

A provider-agnostic swarm of LLM agents that builds, tests, and reviews software.

Project	CodeSwarm — provider-agnostic multi-agent SDLC framework
Version	0.2.0 (draft)
Author	Ankur Srivastava
Date	30 July 2026
Repository	github.com/ankssri/codingswarm
Status	Implemented & verified

Contents

- 1 Executive Summary
- 2 Objectives & Requirements
- 3 Design Principles
- 4 High-Level Architecture
- 5 The Agent Swarm
- 6 The Quality Gates
- 7 Per-Feature Build Loop
- 8 Execution Model: Parallel Build & Merge
- 9 LLM Provider Abstraction
- 10 Tools and the MCP Boundary
- 11 Observability & Artifacts
- 12 Configuration
- 13 Security & Privacy
- 14 Alignment with Anthropic Best Practices
- 15 Limitations & Future Work
- Appendix A Repository Layout
- Appendix B CLI Reference

1 Executive Summary

CodeSwarm is a standalone, vendor-neutral framework that automates the full software development lifecycle using a coordinated **swarm of LLM agents**. From a one-line idea or a short spec it produces a working, tested software project: it gathers requirements, designs an architecture, breaks the work into features, then builds, tests, and reviews each feature until it passes a strict quality bar.

Its defining property is that **quality is verified, not asserted**. A feature is only marked *done* when its tests actually execute and pass, an independent reviewer approves the code, and a dedicated security reviewer clears it. Failing features are reported, never silently shipped.

The framework runs as a plain Python program with **no dependency on Claude Code or any single LLM vendor**. It supports BytePlus ModelArk, OpenAI, and Google Gemini through one OpenAI-compatible client, plus an offline mock provider for demos and CI.

The core guarantee

For a project with N features, CodeSwarm returns a per-feature table of tests / review / security outcomes backed by a real test run — turning "the model generated some code" into "each feature is built and verified."

2 Objectives & Requirements

Business objectives

- Replace manual, step-by-step SDLC execution with a repeatable automated pipeline.
- Remove single-vendor lock-in: run the same swarm on BytePlus, OpenAI, or Gemini.
- Guarantee that generated features are individually built and tested without bugs.
- Be operable by anyone with Python and an API key — no proprietary runtime.

Functional requirements

- Ingest a product idea (inline or YAML spec) and drive it through requirements → design → planning → build/test/review → integration.
- Enforce automated quality gates per feature with a bounded retry budget.
- Persist the artifacts of every phase to disk for inspection and audit.
- Expose a simple CLI and a diagnostic command.

Non-functional requirements

- Provider-agnostic; adding a provider is a small, localized change.
- Deterministic offline mode for testing without API keys.
- Minimal runtime dependencies; generated projects stay dependency-light.
- Transparent, observable execution.

3 Design Principles

Principle	How CodeSwarm applies it
Verify, don't trust	Tests are executed in a subprocess as an objective gate; a reviewer and a security reviewer add independent judgment.
Simple core, smart edges	Orchestration is deterministic Python; intelligence lives in swappable agents, prompts, and tools.
Vendor neutrality	One OpenAI-compatible client behind a tiny provider interface; per-role provider/model overrides.
Isolation for safety	Parallel features build in isolated workspaces; a merge gate re-verifies the whole.
Observability by default	Every phase writes human-readable artifacts as it completes.

4 High-Level Architecture

CodeSwarm is an **orchestrated multi-agent system**, not a linear script. A deterministic orchestrator drives an idea through planning, then hands each feature to a build loop where specialist agents write, test and review the code until it clears three quality gates. Eight specialist agents perform the roles; every agent calls an LLM through one OpenAI-compatible provider, and the agentic Developer and Tester act on the real project through local tools backed by a guarded Workspace and a pytest Sandbox.

Solution overview

Planning is a short single-shot chain; the build stage fans features out to isolated workspaces, then a merge gate re-verifies the whole before the Integrator assembles the output project.

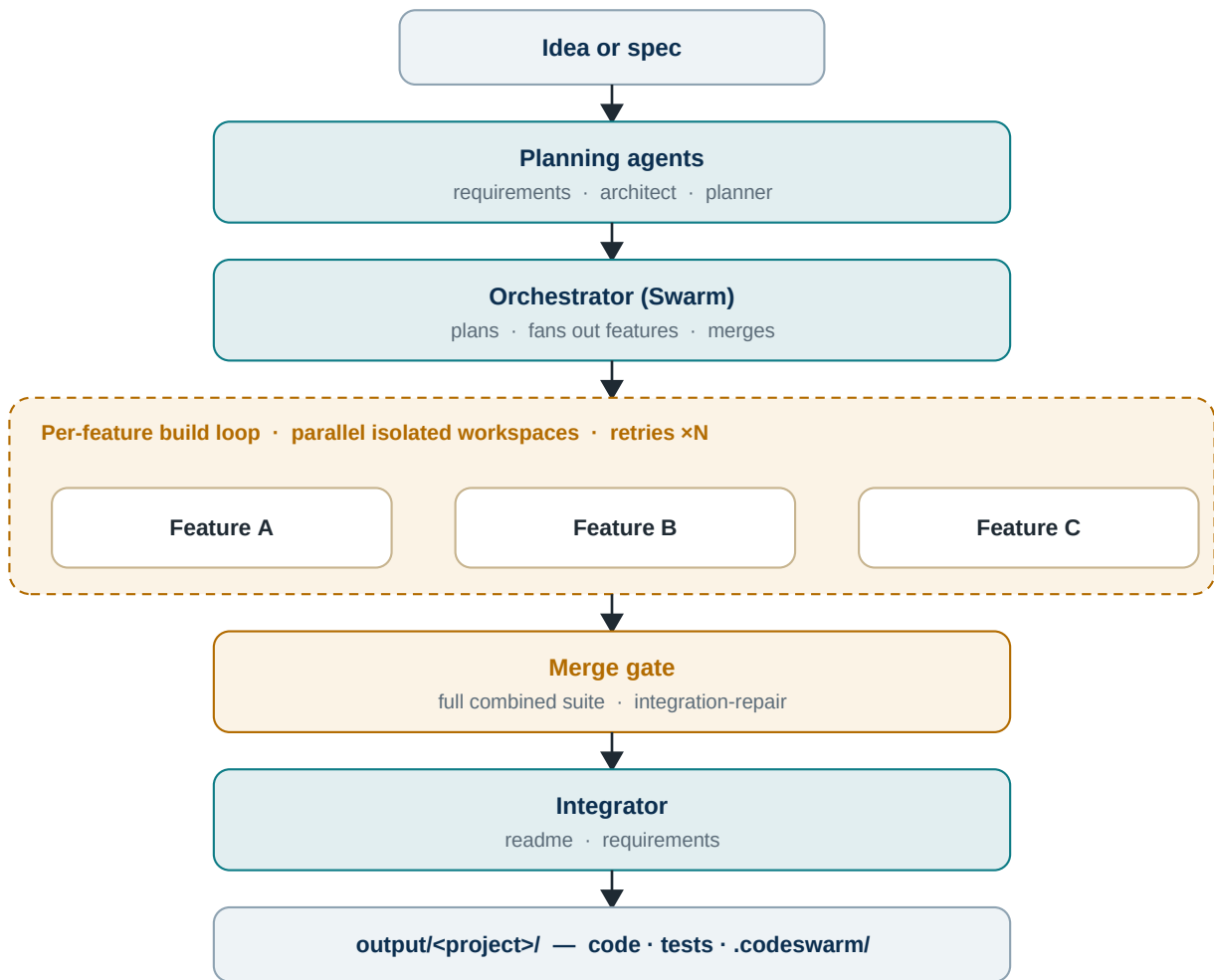


Figure 1 — End-to-end pipeline. Neutral = data & I/O, teal = agents & orchestration, amber = the build / test / review loop.

Loop engineering

The heart of the system is two nested feedback loops. The **inner** loop is the agentic tool-use cycle — the Developer and Tester reason, call a tool, observe the result, and repeat against the live workspace. The **outer** loop is the quality gate: pytest, a correctness reviewer and a security reviewer must all pass, or their failures are compiled into feedback and fed back for a bounded retry (**max_feature_iterations**, default 3).

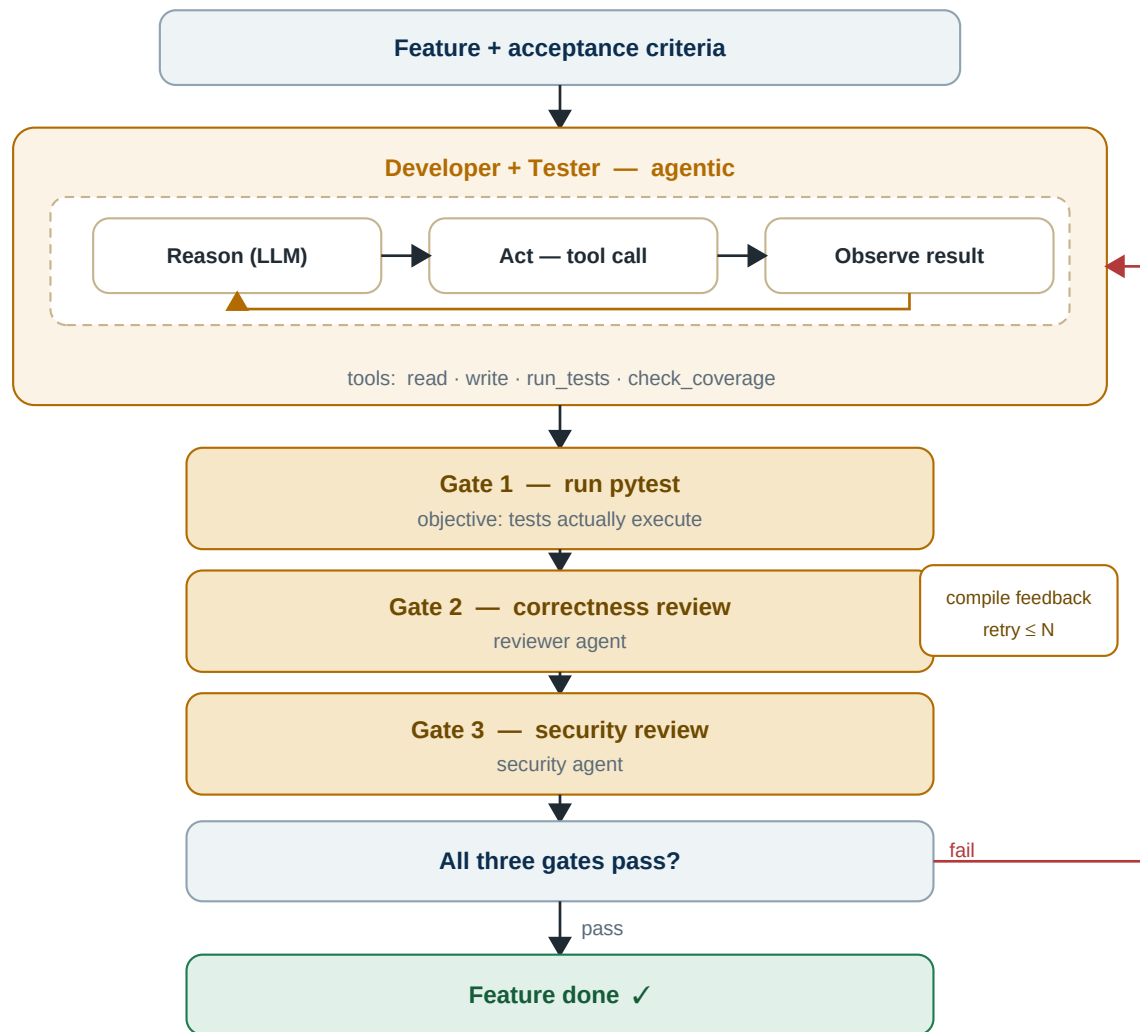


Figure 2 — Nested loops. The inner reason → act → observe cycle runs inside the outer build → three-gate → retry loop; a feature is done only when all three gates pass.

Why Python

Python is the native language of LLM tooling and — crucially — lets the swarm run the tests it writes in-process. That is what makes the quality gate real rather than cosmetic.

5 The Agent Swarm

Each agent is a thin specialist over a provider. Requirements, Architect, Planner, Reviewer, Security Reviewer and Integrator are single-shot; the Developer and Tester are **agentic** — they use tools in a loop to inspect, edit and verify the real project.

Agent	Type	Responsibility
Requirements	single-shot	Turn the idea into concrete, testable requirements.
Architect	single-shot	Produce a minimal technical design and file layout.
Planner	single-shot	Break the design into independently testable features + acceptance criteria.

Agent	Type	Responsibility
Developer	agentic loop	Read/write files and run tests via tools; self-correct until the feature works.
Tester	agentic loop	Write pytest tests and use coverage to close gaps.
Reviewer	single-shot	Correctness gate — approve only if correct and adequately tested.
Security Reviewer	single-shot	Security gate — injection, unsafe eval, path traversal, secrets, etc.
Integrator	single-shot	Assemble README and requirements for the generated project.

6 The Quality Gates

For every feature the orchestrator runs a build/test/review loop, up to **max_feature_iterations** (default 3). A feature is DONE only when **all three** gates pass; otherwise the failures are compiled into feedback and fed back for another attempt.

```

Developer AGENT implements/fixes (reads, writes, runs tests)
  |
  v
Tester AGENT writes tests + checks coverage
  |
  v
GATE 1 (objective) : run pytest      --- fail ----+
  | pass                                         |
  v                                         |
GATE 2 (correctness) : reviewer agent  -- reject --+
  | approve                                     |
  v                                         |
GATE 3 (security)    : security agent  -- block ---+
  | clear                                                    |
  v                                                    |
Feature = DONE      compile feedback -> next attempt (<= 3)

```

If a feature never satisfies all three gates within the retry budget it is marked **FAILED** and surfaced in the report — the swarm never ships an unverified feature.

7 Per-Feature Build Loop

The Developer is a genuine tool-using agent (mirroring how modern coding agents work) rather than a one-shot code generator. It is given a toolbox bound to the feature's workspace:

Tool	Purpose
list_files	Enumerate the current project files.
read_file	Read a file before changing it.
write_file	Create or overwrite a file with full contents.
run_tests	Execute pytest and read the output — self-check.
check_coverage	(Tester) find uncovered lines to add tests for.
finish	Signal the feature is implemented and passing.

This design also solves context management: the agent reads only what it needs from disk rather than being handed the entire codebase as text, which scales far better.

8 Execution Model: Parallel Build & Merge

By default, independent features build **concurrently**, each in its own isolated scratch workspace (via a thread pool, up to **max_parallel_features**). Because agents are stateless per call and the Tester/Reviewer take files as arguments, parallel execution is race-free. After the barrier, all features are merged into the main workspace and the **full combined suite** runs. Any cross-feature regression triggers an integration-repair loop that puts the agentic developer on the merged project until the whole suite is green.

```
plan ----> [feature A]  build/test/review  (isolated dir) --+
          +--> [feature B]  build/test/review  (isolated dir) --+
          +--> [feature C]  build/test/review  (isolated dir) --+
                                     |
                                     v
                        merge all --> run FULL pytest suite (regression gate)
                                     |
                                     v
                    pass  OR  integration-repair loop --> pass
```

Trade-off & safety net

Parallel is faster; sequential (--sequential) lets each feature see prior code as it builds. Same-path conflicts across parallel features are last-writer-wins — the integration-repair loop, not a 3-way merge, is the safety net. In practice each feature owns distinct modules.

9 LLM Provider Abstraction

All supported providers speak the OpenAI chat + tool-calling protocol, so a single **OpenAICompatibleProvider** handles them by swapping base URL and key. Roles can override provider/model independently — e.g. a stronger model for review, a cheaper one for the developer loop, or a different model reviewing than writing.

Provider	Endpoint style	Key (env)
BytePlus ModelArk	ark.ap-southeast.bytepluses.com/api/v3	ARK_API_KEY
OpenAI	api.openai.com/v1	OPENAI_API_KEY
Google Gemini	.../v1beta/openai/ (OpenAI-compatible)	GEMINI_API_KEY
mock	in-process, offline	— (none)

The interface is a single method, so a new provider is one small class. The provider layer also sanitizes keys and makes **.env** authoritative over stale shell exports.

10 Tools and the MCP Boundary

The Developer/Tester tools are **local, in-process Python functions** bound to the workspace — deliberately not MCP. The inner build loop runs on the same machine, so wrapping it in an external protocol would only add latency and failure surface.

MCP is reserved for external systems the code must be grounded in — live API docs, a real database or vector-DB schema, ticketing systems, GitHub, or a staging environment. Such a capability is added as an extra tool whose handler wraps an MCP client call; the agent loop treats it identically to a local tool. For typical application logic the swarm needs no external connection to design, write and test the code.

11 Observability & Artifacts

Every phase writes its output to the project's `.codeswarm/` folder *as it completes*, so progress can be inspected (or tailed) during a run, not only at the end. A `--verbose` flag also streams requirements, design and acceptance criteria to the console.

Artifact	Written after	Contents
<code>requirements.md / .json</code>	Phase 1	Testable functional & non-functional requirements.
<code>design.md</code>	Phase 2	The architecture and module layout the swarm chose.
<code>plan.md / .json</code>	Phase 3	Feature breakdown with acceptance criteria.
<code>report.json</code>	every phase	Requirements, features, per-feature attempts and gate results.
<code>app/ , tests/</code>	Phase 4	The generated source and its test-suite.

12 Configuration

A single YAML file (`config/default.yaml`) drives everything; CLI flags override it. Key knobs:

- **provider / providers** — default provider and each backend's `base_url`, key env, model.
- **agents** — optional per-role provider / model / temperature overrides.
- **swarm** — `max_feature_iterations`, `developer_max_steps`, `parallel_features` & `max_parallel_features`, `run_tests`, `test_timeout_seconds`, `temperature`, `output_dir`, `target_language`.

13 Security & Privacy

- **Secrets stay out of prompts:** API keys are used only as the HTTP auth header, never placed in any message sent to a model. `.env` is git-ignored and never committed.
- **Key hygiene:** keys are sanitized (whitespace/quotes/Bearer); `.env` overrides stale shell exports; auth errors show a masked fingerprint, never the secret.
- **Generated-code execution:** the test sandbox runs generated code in a subprocess with a timeout. For untrusted specs, run inside a container or disposable VM.
- **Security gate:** a dedicated reviewer inspects each feature for common vulnerability classes.

14 Alignment with Anthropic Best Practices

CodeSwarm's spine follows Anthropic's published guidance on building effective agents and multi-agent systems.

Practice	Status	In CodeSwarm
Prompt chaining with gates	Yes	requirements → design → plan → build, gated between steps.
Evaluator–optimizer loop	Yes	developer generates; tests + reviewers evaluate and feed back.
Ground-truth feedback	Yes	pytest is really executed as the objective gate.
Orchestrator–workers	Yes	orchestrator delegates features to isolated workers.
Parallel subagents	Yes	independent features build concurrently, then merge.
Tool-using agent loop	Yes	developer/tester inspect, edit and test via tools.
Independent review	Yes	reviewer/security can run on a different model than the writer.

15 Limitations & Future Work

- Parallel same-file merges are last-writer-wins; a true 3-way merge is future work.
- No durable checkpoint/resume yet — a crashed run restarts (state is serializable, so this is tractable).
- Python is the only target language wired to the test runner; other languages generate code but skip execution.
- Reviewer rubric is prose; explicit multi-dimension rubrics and a performance gate are candidates.
- No token-budget accounting; multi-agent runs are token-intensive by nature.

A Appendix — Repository Layout

```

codeswarm/
  cli.py          command-line interface (build, doctor)
  config.py       layered configuration loading
  pipeline.py     orchestrator + per-feature quality loop   (core)
  models.py      Feature / ProjectState dataclasses
  workspace.py   guarded file writes + snapshots
  sandbox.py     runs pytest + coverage (objective gate)
  tools.py       local dev/tester tools (read/write/run/coverage)
  utils.py       JSON + file-block parsing
  llm/           provider abstraction + tool-calling
  agents/        specialist roles + prompts
  config/default.yaml  examples/  scripts/run_eval.sh  docs/  tests/

```

B Appendix — CLI Reference

Command	Purpose
codeswarm doctor	Check config, API key (masked), connectivity, and tool-calling.
codeswarm build --idea "..."	Build from a one-line idea.
codeswarm build --spec f.yaml	Build from a YAML spec.
--provider / --model	Choose backend and model.

Command	Purpose
--sequential / --max-parallel N	Control feature concurrency.
--max-iterations N	Retry budget per feature.
--verbose / -v	Stream requirements, design, and criteria live.
--dry-run	Offline mock provider (no API key).

© 2026 Ankur Srivastava · CodeSwarm is released under the MIT License. This document describes the design as implemented at version 0.2.0 (draft).